غزل ملانوروزی

پروژه درس موبایل ۲

موضوع بازی سودوکو

۱. نمای کلی و معماری (High-Level Overview)

«پروژه ما یک اپلیکیشن سودوکو است که با معماری **Decoupled (جدا شده)** طراحی شده است. یعنی منطق ریاضی بازی (Logic) کاملاً از ظاهر برنامه (UI) جداست. این کار باعث می‌شود اگر بخواهیم فردا ظاهر برنامه را از CustomTkinter به یک وب‌سایت یا اپلیکیشن موبایل تغییر دهیم، نیازی به تغییر حتی یک خط از کدهای ریاضی و الگوریتم‌ها نداشته باشیم.»

اجزای اصلی سیستم:

۱. **مدیر جدول (SudokuBoard):** مسئول خلق جهانِ بازی (تولید پازل).

۲. **مغز متفکر (SudokuSolver):** مسئول حل کردن پازل با هوش مصنوعی.

۳. **داور (SudokuValidator):** مسئول بررسی قوانین و اعلام برنده یا بازنده.

٤. **رابط کاربری (SudokuApp):** مسئول تعامل با کاربر و نمایش گرافیکی.

---

۲. تحلیل دقیق متدها و توابع (Deep Dive)

الف) کلاس SudokuBoard (تولیدکننده پازل)

هدف: ساخت یک جدول ۹×۹ که طبق قوانین سودوکو باشد و سپس خالی کردن برخی خانه‌ها.

- **__init__**: یک ماتریس ۹×۹ از اعداد صفر ایجاد می‌کند. صفر یعنی خانه خالی.
- **generate_puzzle(empty_cells)**:
- ابتدا یک جدول کاملاً پر و درست می‌سازد.
- سپس لیست تمام خانه‌ها را می‌گیرد و آن‌ها را shuffle (بُر زدن) می‌کند.

- در نهایت به تعداد empty_cells که کاربر مشخص کرده، خانه‌ها را صفر می‌کند تا پازل ساخته شود.
- **_(fill_board_recursive(board** (**الگوریتم اصلی**):
- این یک تابع **Recursive** (بازگشتی) است.
- تابع ابتدا یک خانه خالی پیدا می‌کند. سپس اعداد ۱ تا ۹ را به صورت تصادفی امتحان می‌کند.
- اگر عددی در آن خانه "امن" (is_safe) باشد، آن را قرار داده و خودش را دوباره صدا می‌زند تا خانه بعدی را پر کند.
- **نکته فنی:** اگر به جایی برسد که هیچ عددی جایگزین نشود، تابع مقدار False برمی‌گرداند، یعنی در این مسیر به بن‌بست خورده‌ایم. در این حالت، عدد قبلی را پاک کرده و به مرحله قبل برمی‌گردیم تا مسیر دیگری را تست کنیم.
- **_(is_safe(board, r, c, num**: قلب منطق ریاضی. چک می‌کند که عدد مورد نظر در سطر r ، ستون c و آن بلوک ۳×۳ خاص، تکراری نباشد.

ب) کلاس SudokuSolver (حل‌کننده خودکار)

هدف: پیدا کردن پاسخ نهایی پازل در کمترین زمان.

- **solve(board)**: یک کپی از جدول اصلی می‌گیرد تا جدول اصلی کاربر دست‌نخورده بماند و سپس فرآیند حل را شروع می‌کند.
- **_(backtrack(board**:
- این دقیقاً مشابه الگوریتم تولید پازل است، اما با یک هدف متفاوت.

- این تابع با استفاده از تکنیک **Depth-First Search** (**جستجوی اول سطح**)، تمام احتمالات ممکن را بررسی می‌کند تا به حالتی برسد که هیچ خانه خالی‌ای در جدول باقی نمانده باشد.

ج) کلاس SudokuValidator (داور بازی)

هدف: تایید نهایی پاسخ کاربر.

- **is_valid_solution(board)**: یک زنجیره بررسی است. ابتدا چک می‌کند جدول کامل است (_is_complete) و سپس با صدا کردن متدهای سطر، ستون و بلوک، صحت کل جدول را تایید می‌کند.
- **is_valid_row/col/block_**: این متدها از یک ترفند پایتونی استفاده می‌کنند: تمام اعداد یک سطر را برمی‌دارند، آن‌ها را در یک set می‌ریزند (که در مجموعه، تکراری وجود ندارد) و چک می‌کنند که آیا تعداد اعضای set با تعداد اعداد اصلی برابر است یا خیر. اگر برابر نباشد، یعنی تکرار وجود داشته و قانون نقض شده است.

د) کلاس SudokuApp (رابط کاربری – UI)

هدف: تبدیل کدها به دکمه، رنگ و متن.

- **validate_cell_realtime(r, c)**: این متد برای ایجاد تجربه کاربری (UX) عالی است. به محض اینکه کاربر عددی را تایپ می‌کند (KeyRelease)، این تابع اجرا شده و آن خانه را بررسی می‌کند. اگر عدد تکراری باشد، رنگ پس‌زمینه خانه را قرمز می‌کند.

- **solve_game**(): این متد پلی میان UI و Solver است. اطلاعات را از خانه‌های متنی (Entry) می‌گیرد، به SudokuSolver می‌دهد و پاسخ را دوباره در خانه‌هایی که کاربر پر نکرده، نمایش می‌دهد.

---

۳. چرخه حیات برنامه (The Execution Workflow)

در این بخش توضیح می‌دهید که از لحظه کلیک بر روی main.py تا پایان بازی چه اتفاقی می‌افتد.

۱. **شروع (Startup)**: کاربر main.py را اجرا می‌کند. کلاس SudokuApp ساخته می‌شود.

۲. **آماده‌سازی (Initialization)**:

- SudokuApp یک شیء از SudokuBoard می‌سازد.
- SudokuBoard ابتدا یک جدول کامل تولید می‌کند (_fill_board_recursive).
- سپس با حذف خانه‌ها، پازل را آماده می‌کند.
- این پازل به گرید (Grid) در رابط کاربری منتقل می‌شود.

۳. **تعامل کاربر (Gameplay)**:

- کاربر شروع به وارد کردن اعداد می‌کند.
- هر بار که عددی وارد می‌شود، رویداد KeyRelease باعث اجرای validate_cell_realtime می‌شود تا خطاها همان لحظه مشخص شوند.

٤. **پایان (Resolution)**:

- یا کاربر با وارد کردن اعداد صحیح، برنده می‌شود (که توسط SudokuValidator تایید می‌شود).
- یا کاربر از دکمه “Solve” استفاده می‌کند تا الگوریتم Backtracking کل پازل را در کسری از ثانیه حل کرده و پاسخ را نمایش دهد.